

TECHNICAL APPENDIX

PIPELINE DESIGN AND REPRODUCIBILITY NOTES

UTD Edge Fusion Preliminary Experiment

Scope

This document records the engineering design of the uploaded experiment pipeline. It covers data scanning, feature extraction, clean-condition baselines, degraded-sensing tests, optional Depth/RGB extensions, generated outputs, and local reproduction steps.

Document metadata

Pipeline name	UTD Edge Fusion Preliminary Experiment
Code package	utd_edge_fusion_exp
Main modalities	Skeleton, Inertial
Optional modalities	Depth, RGB/video
Main tests	Clean comparison; degraded-sensing robustness
Python environment	Python 3.10–3.11 recommended; install packages from <code>requirements.txt</code>
Date	April 2026

Contents

1	Document Boundary	1
1.1	Purpose	1
1.2	Code Package Covered	1
2	Pipeline Overview	2
2.1	High-Level Flow	2
2.2	Experiment Question	2
3	Data and Configuration Design	3
3.1	Expected Data Layout	3
3.2	Core Configuration	3
3.3	Output Directories	3
4	Module Design	5
4.1	Utility Modules	5
4.2	Executable Scripts	5
5	Feature and Fusion Design	6
5.1	Time-Series Feature Extraction	6
5.2	Degradation Simulation	6
5.3	Reliability-Aware Representation	7
6	Training and Evaluation Design	8
6.1	Model Choices	8
6.2	Metrics	8
6.3	Clean-Condition Comparison	8
6.4	Degraded-Sensing Robustness	9
6.5	Extended Optional Modalities	9

7	Commands and Reproduction	10
7.1	Installation	10
7.2	Set Data Root	10
7.3	Core Pipeline	10
7.4	Optional Extended Pipeline	11
7.5	Expected Console Messages	11
8	Generated Files	13
8.1	Core Outputs	13
8.2	Extended Outputs	13
8.3	Payload Estimate	14
9	Attention Points and Troubleshooting	15
9.1	Checks Before Running	15
9.2	Common Failure Cases	15
9.3	Limitations of the Current Implementation	16
A	Core Execution Structure	17
A.1	Core Runner	17
A.2	Optional Runner	17

1 Document Boundary

1.1 Purpose

This document describes the uploaded experiment code from an engineering perspective. It records the pipeline structure, module responsibilities, required inputs, generated outputs, and local reproduction steps.

The scope is limited to the current research prototype: a UTD-style multimodal action-data pipeline for clean-condition and degraded-sensing comparisons.

Boundary
The current code is a research prototype. It is suitable for preliminary feasibility testing and method comparison. It is not a production deployment package, a full real-time edge system, or a complete benchmark suite.

1.2 Code Package Covered

The document covers the following package:

```
1 utd_edge_fusion_exp/  
2   config.py  
3   utils_io.py  
4   utils_features.py  
5   utils_models.py  
6   00_check_env.py  
7   01_scan_dataset.py  
8   02_extract_features.py  
9   03_train_clean_baselines.py  
10  04_eval_degraded_sensing.py  
11  05_export_tables_figures.py  
12  06_optional_extract_depth_rgb_features.py  
13  07_train_extended_modalities.py  
14  08_eval_extended_degraded_robustness.py  
15  run_all_core.py  
16  run_extended.py  
17  requirements.txt  
18  README.md
```

2 Pipeline Overview

2.1 High-Level Flow

The pipeline follows a staged structure. Each script writes intermediate files to `outputs/`, so later stages can be rerun without repeating every earlier step.

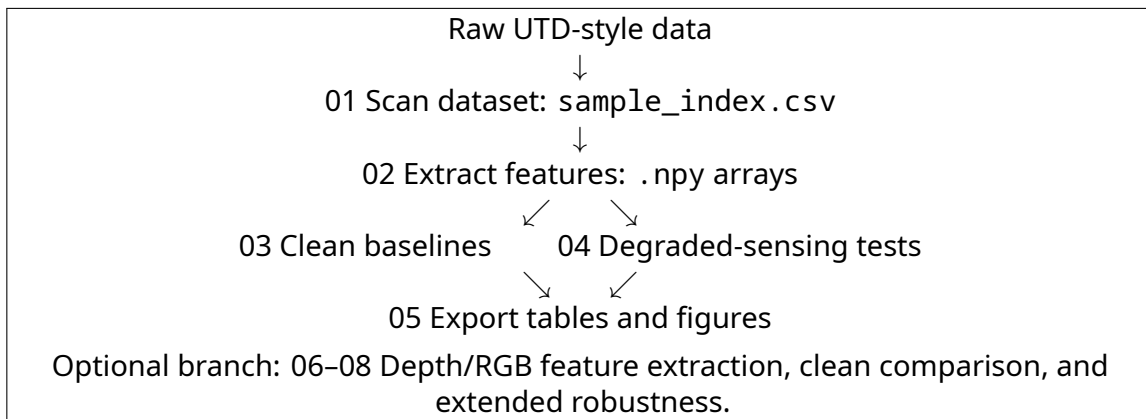


Figure 2.1: Pipeline flow implemented by the uploaded scripts.

2.2 Experiment Question

The implemented experiment checks a focused question: after extracting lightweight features from multiple sensor streams, how do single-modality models, simple early fusion, and reliability-aware fusion behave under clean and degraded sensing conditions?

This is tested with:

- Skeleton-only baseline;
- Inertial-only baseline;
- Early fusion by concatenating feature vectors;
- Reliability-aware fusion, where feature vectors are weighted by simulated reliability indicators;
- Optional extended runs with Depth and RGB/video features, if those files are available.

3 Data and Configuration Design

3.1 Expected Data Layout

The default data root is defined in `config.py`. It can also be changed with the `UTD_DATA_ROOT` environment variable.

```
1 DATASET/  
2   Skeleton/  
3     a<action>_s<subject>_t<trial>_skeleton.mat  
4   Inertial/  
5     a<action>_s<subject>_t<trial>_inertial.mat  
6   Depth/                               # optional  
7     a<action>_s<subject>_t<trial>_depth.mat  
8   ... RGB/video files ... # optional .avi / .mp4 / .mov
```

The parser accepts strict names for Skeleton, Inertial and Depth .mat files. RGB/video file names are allowed to be less strict, as long as the action, subject and trial pattern can still be detected.

3.2 Core Configuration

The main experiment settings are centralised in `config.py`. The subject split, random seeds, degradation rates and model family should be checked before running experiments.

```
1 TRAIN_SUBJECTS = {1, 2, 3, 4}  
2 TEST_SUBJECTS = {5, 6, 7, 8}  
3 RANDOM_SEEDS = [11, 22, 33, 44, 55]  
4 DEGRADATION_RATES = [0.0, 0.1, 0.3, 0.5]  
5 MODEL_TYPE = "rf" # "rf" or "lr"  
6 ENABLE_DEPTH_FEATURES = False  
7 ENABLE_RGB_FEATURES = False
```

Listing 3.1: Core settings in `config.py`

The split is subject-based rather than random sample-based. This is useful for checking generalisation to unseen subjects, although it also means that results can be sensitive to the selected subject groups.

3.3 Output Directories

The package creates the output folders automatically when `config.py` is imported.

```
1 outputs/  
2   index/  
3   features/  
4   results/  
5   figures/
```

This simple layout keeps intermediate arrays, result tables and figures separated.

4 Module Design

4.1 Utility Modules

File	Role
<code>utils_io.py</code>	Parses sample file names and loads variables from MATLAB .mat files. It accepts strict Skeleton/Inertial/Depth names and optional RGB/video file patterns.
<code>utils_features.py</code>	Converts raw time series into statistical feature vectors. It also implements degradation simulation and reliability-aware feature construction.
<code>utils_models.py</code>	Builds either a random forest classifier or a logistic-regression pipeline, evaluates accuracy and macro-F1, and summarises multi-seed results.

4.2 Executable Scripts

Script	Main input	Main output / function
<code>00_check_env.py</code>	Local Python environment	Prints Python version, data path, output path and package availability.
<code>01_scan_dataset.py</code>	Raw dataset folders	Writes <code>outputs/index/sample_index.csv</code> .
<code>02_extract_features.py</code>	<code>sample_index.csv</code>	Writes Skeleton/Inertial feature arrays and label/subject arrays.
<code>03_train_clean_baselines.py</code>	Feature arrays	Trains clean-condition baselines and writes clean result tables.
<code>04_eval_degraded_sensing.py</code>	Feature arrays	Simulates sensing degradation and writes robustness result tables.
<code>05_export_tables_figures.py</code>	Result CSV files	Exports LaTeX tables, robustness curves and a payload estimate.
<code>06_optional_extract_depth_rgb_features.py</code>	Indexed optional paths	Extracts Depth/RGB features when optional files are available.
<code>07_train_extended_modalities.py</code>	Core and optional features	Compares clean-condition settings with optional modalities.
<code>08_eval_extended_degraded_robustness.py</code>	Core and optional features	Compares early, augmented and reliability-aware fusion under degradation.
<code>run_all_core.py</code>	Scripts 00–05	Runs the core sequence and stops on the first failed step.
<code>run_extended.py</code>	Scripts 06–08	Runs the optional Depth/RGB extension sequence.

5 Feature and Fusion Design

5.1 Time-Series Feature Extraction

The feature extractor uses summary statistics rather than a deep sequence encoder. For each time-series dimension, it computes values such as mean, standard deviation, minimum, maximum, median, quartiles, RMS, mean absolute value and energy. It also adds statistics from first-order temporal differences.

```
1 def safe_stats_1d(x):
2     x = np.asarray(x, dtype=np.float64)
3     if x.size == 0:
4         return np.zeros(10, dtype=np.float64)
5     x = np.nan_to_num(x, nan=0.0, posinf=0.0, neginf=0.0)
6     return np.array([
7         np.mean(x), np.std(x), np.min(x), np.max(x), np.median(x),
8         np.percentile(x, 25), np.percentile(x, 75),
9         np.sqrt(np.mean(np.square(x))), np.mean(np.abs(x)),
10        np.sum(np.square(x)) / max(len(x), 1),
11    ], dtype=np.float64)
```

Listing 5.1: Feature statistics used in utils_features.py

Skeleton arrays are expected as `d_skel` with shape (20 joints, 3 coordinates, T frames). The code transposes them into time-major format and centres joints per frame before feature extraction. Inertial arrays are expected as `d_iner` with shape (T, 6).

5.2 Degradation Simulation

Degraded sensing is simulated in feature space. At a given degradation rate, the function adds scaled Gaussian noise and randomly sets feature values to zero.

```
1 def degrade_features(X, rate, rng, noise_scale=0.05):
2     X = np.asarray(X, dtype=np.float64)
3     if rate <= 0:
4         return X.copy()
5     Xd = X.copy()
6     std = np.std(Xd, axis=0, keepdims=True)
7     std[std == 0] = 1.0
8     Xd = Xd + rng.normal(0, noise_scale * rate, size=Xd.shape) * std
9     mask = rng.random(size=Xd.shape) < rate
10    Xd[mask] = 0.0
11    return Xd
```

Listing 5.2: Feature-level degradation simulation

This is a controlled stress test, not a full physical sensor corruption model. It is still useful for comparing whether fusion strategies collapse quickly when the feature stream becomes noisier or partially missing.

5.3 Reliability-Aware Representation

The core reliability-aware feature vector scales Skeleton and Inertial features by estimated reliability values and appends the reliability/degradation indicators.

```

1  def build_reliability_features(Xs, Xi, skel_rate, iner_rate):
2      r_s, r_i = 1.0 - skel_rate, 1.0 - iner_rate
3      indicators = np.tile(
4          np.array([r_s, r_i, skel_rate, iner_rate], dtype=np.float64),
5          (Xs.shape[0], 1)
6      )
7      return np.hstack([r_s * Xs, r_i * Xi, indicators])

```

Listing 5.3: Core reliability-aware fusion representation

The extended script generalises the same idea to an arbitrary list of available modalities:

$$X_{fusion} = [r_1 X_1, r_2 X_2, \dots, r_m X_m, r_1, \dots, r_m, d_1, \dots, d_m].$$

In the current implementation, optional modalities share the same simulated degradation rate during the extended robustness test.

6 Training and Evaluation Design

6.1 Model Choices

The classifier is selected through `MODEL_TYPE`. The default option is a random forest with class-balanced subsampling. The alternative is a standardised logistic-regression pipeline with class balancing.

```
1 def make_model(seed=0):
2     if MODEL_TYPE == "lr":
3         return make_pipeline(
4             StandardScaler(),
5             LogisticRegression(max_iter=2000, class_weight="balanced", random_state=
6                 seed)
7         )
8     return RandomForestClassifier(
9         n_estimators=400, random_state=seed,
10        class_weight="balanced_subsample", n_jobs=-1
11    )
```

Listing 6.1: Model selection logic in `utils_models.py`

6.2 Metrics

The current implementation reports:

- **Accuracy:** overall proportion of correct predictions;
- **Macro-F1:** average F1 across classes without weighting by class size.

Macro-F1 is important when classes are not evenly represented. Future versions can add confusion matrices, per-class recall, calibration error and confidence intervals if the experiment needs more detailed reporting.

6.3 Clean-Condition Comparison

The clean-condition script evaluates four core settings:

- Skeleton-only;
- Inertial-only;

- Early fusion by `np.hstack([Xs, Xi]);`
- Reliability-aware fusion with clean reliability indicators.

The script repeats training over five seeds and saves both raw seed-level results and grouped summaries.

6.4 Degraded-Sensing Robustness

The degraded script trains single-modality and early-fusion models on clean data. It then evaluates them on degraded test features. The reliability-aware model is trained with degraded augmentation across all configured degradation rates.

Interpretation
The robustness result should be read as a relative comparison across model strategies and degradation rates. It does not claim to reproduce every real sensor failure mode.

6.5 Extended Optional Modalities

The optional extension checks whether Depth and RGB features are available and alignable with the core sample IDs. If alignment fails, the optional modality is skipped instead of silently mixing unmatched samples.

Clean extended comparison can include:

- Depth-only or RGB-only;
- Skeleton+Inertial+Depth;
- Skeleton+Inertial+RGB;
- Depth+RGB;
- all available modalities.

The extended degraded script compares:

- early fusion trained on clean features;
- augmented early fusion trained with degraded features;
- reliability-aware fusion trained with degradation augmentation and reliability indicators.

7 Commands and Reproduction

7.1 Installation

The required packages are listed in `requirements.txt`.

```
1 pip install -r requirements.txt
```

Listing 7.1: Basic installation

RGB/video feature extraction requires OpenCV. It is optional for the core Skeleton+Inertial experiment.

```
1 pip install opencv-python
```

Listing 7.2: Optional RGB/video dependency

7.2 Set Data Root

On Windows Command Prompt:

```
1 set UTD_DATA_ROOT=C:\path\to\UTD_dataset\DATASET
```

On PowerShell:

```
1 $env:UTD_DATA_ROOT="C:\path\to\UTD_dataset\DATASET"
```

On macOS or Linux:

```
1 export UTD_DATA_ROOT=/path/to/UTD_dataset/DATASET
```

If the environment variable is not set, the code uses the default path in `config.py`.

7.3 Core Pipeline

The core pipeline can be run step by step:

```
1 python 00_check_env.py
2 python 01_scan_dataset.py
3 python 02_extract_features.py
4 python 03_train_clean_baselines.py
5 python 04_eval_degraded_sensing.py
6 python 05_export_tables_figures.py
```

Or through the core runner:

```
1 python run\_all\_core.py
```

7.4 Optional Extended Pipeline

After the core feature extraction succeeds, optional Depth/RGB experiments can be run:

```
1 python 06_optional_extract_depth_rgb_features.py
2 python 07_train_extended_modalities.py
3 python 08_eval_extended_degraded_robustness.py
```

Or through the extended runner:

```
1 python run\_extended.py
```

7.5 Expected Console Messages

The following blocks show the message format printed by the scripts. Numeric values depend on the local dataset and environment.

```
1 Python: <python version>
2 DATA_ROOT: <resolved dataset path>
3 OUT_DIR: <resolved output path>
4
5 Required packages:
6   [OK] numpy: <version>
7   [OK] pandas: <version>
8   [OK] scipy: <version>
9   [OK] sklearn: <version>
10  [OK] matplotlib: <version>
11
12 Optional packages:
13   [OK] cv2: <version>
14   # or: [OPTIONAL MISSING] cv2: <message>
15
16 Environment check passed.
```

Listing 7.3: Environment check message pattern

```
1 Dataset scan complete.
2 Skeleton samples: <n_skeleton>
3 Inertial samples: <n_inertial>
4 Depth samples: <n_depth>
5 Optional RGB videos: <n_rgb>
6 Common skeleton+inertial samples: <n_common>
7 Saved: outputs/index/sample_index.csv
8 <first rows of the sample index>
```

Listing 7.4: Dataset scan message pattern

```
1 Feature extraction complete.
2 Samples: <n_samples>
```

```

3 Skeleton feature shape: (<n_samples>, <skeleton_features>)
4 Inertial feature shape: (<n_samples>, <inertial_features>)
5 Classes: <n_classes>
6 Subjects: [<subject ids>]

```

Listing 7.5: Feature extraction message pattern

```

1 Train samples: <n_train> Test samples: <n_test>
2
3 Clean-condition summary:
4     model  accuracy_mean  accuracy_std  macro_f1_mean  macro_f1_std
5 0      Early fusion      ...           ...           ...           ...
6 1      Inertial-only      ...           ...           ...           ...
7 2 Reliability-aware fusion ...           ...           ...           ...
8 3      Skeleton-only      ...           ...           ...           ...

```

Listing 7.6: Clean-condition result message pattern

```

1 Clean-condition extended modality comparison
2 Train samples: <n_train> Test samples: <n_test>
3 Available modalities: {'Skeleton': (<n>, <d1>), 'Inertial': (<n>, <d2>), ...}
4 Settings:
5   - Skeleton-only: ['Skeleton']
6   - Inertial-only: ['Inertial']
7   - Skeleton+Inertial: ['Skeleton', 'Inertial']
8   - Skeleton+Inertial+Depth: ['Skeleton', 'Inertial', 'Depth']

```

Listing 7.7: Extended modality message pattern

8 Generated Files

8.1 Core Outputs

A successful core run creates these files:

```
1 outputs/  
2   index/sample_index.csv  
3   features/X_skeleton.npy  
4   features/X_inertial.npy  
5   features/y.npy  
6   features/subjects.npy  
7   features/trials.npy  
8   features/sample_ids.csv  
9   results/clean_results_by_seed.csv  
10  results/clean_results_summary.csv  
11  results/degraded_results_by_seed.csv  
12  results/degraded_results_summary.csv  
13  results/table_clean_latex.txt  
14  results/table_degraded_latex.txt  
15  results/payload_estimate.csv  
16  results/table_payload_latex.txt  
17  figures/robustness_curve.pdf  
18  figures/robustness_curve.png
```

8.2 Extended Outputs

The optional pipeline can additionally create:

```
1 outputs/  
2   features/X_depth_optional.npy  
3   features/depth_optional_ids.csv  
4   features/X_rgb_optional.npy  
5   features/rgb_optional_ids.csv  
6   results/extended_clean_results_by_seed.csv  
7   results/extended_clean_results_summary.csv  
8   results/table_extended_clean_latex.txt  
9   results/extended_degraded_results_by_seed.csv  
10  results/extended_degraded_results_summary.csv  
11  results/table_extended_degraded_latex.txt  
12  figures/extended_robustness_core_splusi.pdf  
13  figures/extended_robustness_core_splusi.png  
14  figures/extended_robustness_all_available.pdf  
15  figures/extended_robustness_all_available.png
```



```
16 figures/extended_reliability_comparison.pdf
17 figures/extended_reliability_comparison.png
```

8.3 Payload Estimate

`05_export_tables_figures.py` estimates a simple communication payload difference between raw feature upload and edge event summary upload. The estimate is not a network benchmark; it is a compact design note showing why event-level edge summaries may reduce transmitted data.

9 Attention Points and Troubleshooting

9.1 Checks Before Running

- Confirm `DATA_ROOT` points to the correct dataset folder.
- Confirm the filename pattern includes action, subject and trial numbers.
- Run `00_check_env.py` before running the full pipeline.
- Keep `ENABLE_DEPTH_FEATURES` and `ENABLE_RGB_FEATURES` disabled unless the optional files and dependencies are ready.
- Check whether the subject split is appropriate for the current experiment.

9.2 Common Failure Cases

Symptom	Likely cause	Action
<code>DATA_ROOT</code> does not exist	Wrong default path or missing environment variable	Set <code>UTD_DATA_ROOT</code> or edit <code>config.py</code> .
Missing Skeleton/Inertial samples	Folder name or filename pattern mismatch	Check <code>Skeleton/</code> , <code>Inertial/</code> and filename spelling.
Cannot find <code>d_skel</code> or <code>d_iner</code>	MATLAB variable name differs from expected key	Inspect the <code>.mat</code> file and adjust <code>load_mat_var</code> usage.
Optional RGB skipped	OpenCV missing or video path cannot be read	Install <code>opencv-python</code> and check video file format.
Optional modality skipped	Optional feature IDs cannot align to master sample IDs	Check that optional files use matching action/subject/trial identifiers.
Unexpectedly low score	Small feature set, subject split difficulty, data mismatch or degradation strength	Inspect per-class results and run multiple seeds.

9.3 Limitations of the Current Implementation

- Feature extraction is handcrafted and compact; it does not model full temporal dynamics like an LSTM, Transformer or graph model.
- Degradation is simulated at feature level. It is useful for controlled comparison but does not replace sensor-level corruption modelling.
- The evaluation currently reports accuracy and macro-F1. More diagnostic metrics may be needed for a full study.
- The subject split is fixed in `config.py`. More split designs may be needed if the result is used as formal evidence.
- Optional Depth/RGB features are lightweight summaries. They should be treated as an extension check, not as a final video-understanding model.

A Core Execution Structure

A.1 Core Runner

The core runner executes the scripts in order and stops if a step fails.

```
1 steps = [  
2     "00_check_env.py",  
3     "01_scan_dataset.py",  
4     "02_extract_features.py",  
5     "03_train_clean_baselines.py",  
6     "04_eval_degraded_sensing.py",  
7     "05_export_tables_figures.py",  
8 ]  
9  
10 for step in steps:  
11     print("\n" + "=" * 80)  
12     print("Running", step)  
13     print("=" * 80)  
14     result = subprocess.run([sys.executable, step])  
15     if result.returncode != 0:  
16         print(f"[STOP] {step} failed with code {result.returncode}")  
17         raise SystemExit(result.returncode)
```

Listing A.1: run_all_core.py

A.2 Optional Runner

The optional runner assumes that the core feature extraction has already succeeded.

```
1 for script in [  
2     "06_optional_extract_depth_rgb_features.py",  
3     "07_train_extended_modalities.py",  
4     "08_eval_extended_degraded_robustness.py",  
5 ]:  
6     print("\n" + "=" * 80)  
7     print(f"Running {script}")  
8     print("=" * 80)  
9     subprocess.run([sys.executable, script], check=True)
```

Listing A.2: run_extended.py